



Sky: Diamond PAU v1.11.0

Security Review

Cantina Managed review by:
M4rio.eth, Lead Security Researcher
Xmxanuel, Lead Security Researcher

March 19, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	UniswapV3 <code>removeLiquidity</code> can DoS when accrued fees are large relative to removed principal	4
3.1.2	Current <code>diamond-pau</code> changes that affect Grove	5
3.2	Low Risk	8
3.2.1	Uniswap V3 and Uniswap V4 asymmetries	8
3.2.2	<code>CCTPLib</code> hardcoded <code>MAX_FEE = 0</code> has no on-chain update path	9
3.3	Informational	10
3.3.1	Rate-limit accounting is inconsistent across request/cancel and unwind flows	10
3.3.2	Current <code>diamond-pau</code> changes that affect Spark	11
3.3.3	UniswapV3 <code>removeLiquidity</code> requires an active withdraw limit even for the zero-output side	13
3.3.4	<code>CurveLib.swap</code> <code>minAmountOut</code> check is less conservative due to additional truncation	13
3.3.5	<code>LayerZeroLib</code> : no refund for overpaid <code>nativeFee</code> in ETH	14
3.3.6	Minor issues, Typos & Documentation	15

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Sky Protocol is a decentralised protocol developed around the USDS stablecoin.

From Mar xxst to Apr xxst the Cantina team conducted a review of [diamond-pau](#) on commit hash [6b0958f5](#) (tag [v1.11.0-beta.1](#)). The team identified a total of **10** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	2	2	0
Low Risk	2	2	0
Gas Optimizations	0	0	0
Informational	6	2	4
Total	10	6	4

The Cantina Managed team reviewed Sky's [diamond-pau](#) holistically on commit hash [697a8dbb](#) (tag [v1.11.0](#)) and concluded that all findings were addressed and no new vulnerabilities were identified.

3 Findings

3.1 Medium Risk

3.1.1 UniswapV3 removeLiquidity can DoS when accrued fees are large relative to removed principal

Severity: Medium Risk

Context: [UniswapV3Lib.sol#L389](#)

Description: `UniswapV3Lib.removeLiquidity` reuses the same `min.amount0 / min.amount1` values for two different purposes:

1. As the minimum acceptable output in Uniswap position manager `decreaseLiquidity`, which only returns principal (no fees):

```
nonfungiblePositionManager.decreaseLiquidity(
    INonfungiblePositionManager.DecreaseLiquidityParams({
        tokenId: tokenId,
        liquidity: liquidity,
        amount0Min: min.amount0,
        amount1Min: min.amount1,
        deadline: block.timestamp
    })
);
```

This enforces: $\text{min} \leq R$ (where R is the principal returned).

2. As the minimum expected amount in `_checkSlippage`, which restricts the relayer passed minimum amount against a slippage defined by the admin. This `_checkSlippage` runs after `collect`. Since `collect` sweeps the full balance (principal + accrued fees), the check runs against $R + F$:

```
// After collect sweeps principal + accrued fees into the contract:
_checkSlippage(balanceAfter - balanceBefore, min.amount0, maxSlippage);
```

See `UniswapV3Lib.sol#L881`, which enforces that the caller-provided `min` is not too far below the actual received amount:

```
function _checkSlippage(uint256 amount, uint256 minAmount, uint256 maxSlippage) internal
→ pure {
    if (minAmount < (amount * maxSlippage) / 1e18) {
        revert("UniswapV3Lib/min-amount-below-bound");
    }
}
```

This enforces: $\text{min} \geq (R + F) * \text{slippage}$. The problem is that the same `min` value must satisfy both constraints simultaneously:

- $\text{min} \leq R$ (from `decreaseLiquidity`).
- $\text{min} \geq (R + F) * \text{slippage}$ (from `_checkSlippage`).

When accrued fees F grow large enough relative to principal R , no valid `min` exists. With a typical `maxSlippage` of 98% (0.98), the function reverts once fees exceed ~2% of the removed principal on either token side. The transaction fails with `UniswapV3Lib/min-amount-below-bound` even though the caller set `min` to the maximum value `decreaseLiquidity` allows.

This creates a fee-driven denial of service: positions with sufficiently large accrued fees relative to the principal being removed cannot be unwound through the controller. It would require the admin to reduce the `maxSlippage` value to make a `removeLiquidity` call possible again.

Recommendation: Run the slippage check with the `decreaseLiquidity` return values before `collect`, so it only validates the returned principal amount and is not inflated by accrued fees.

Sky: Fixed in [PR 38](#).

Cantina Managed: Partially fixed. The fix creates a series of asymmetries:

- Asymmetry with Uni v4: Uni v3 now does not counts the fees in the limit, v4 counts the fees in the limit because `DECREASE_LIQUIDITY` collects the fees and in Uni v4 the implementation:

```
uint256 startingBalance0 = _getBalance(token0, proxy);
uint256 startingBalance1 = _getBalance(token1, proxy);

IALMProxy(proxy).doCall(_POSITION_MANAGER, callData);

uint256 endingBalance0 = _getBalance(token0, proxy);
uint256 endingBalance1 = _getBalance(token1, proxy);

uint256 rateLimitDecrease =
    _getNormalizedBalance(token0, endingBalance0 - startingBalance0) +
    _getNormalizedBalance(token1, endingBalance1 - startingBalance1);

// NOTE: Rate limit decrease includes any token0 or token1 received due to fees.
_decreaseRateLimit(rateLimits, LIMIT_WITHDRAW, poolId, rateLimitDecrease);
```

- Uni v4 has one limit per pool

```
_decreaseRateLimit(rateLimits, LIMIT_WITHDRAW, poolId, rateLimitDecrease);
```

While Uni v3 has two limits per pool

```
_decreaseRateLimit(rateLimits, LIMIT_WITHDRAW, token0, pool, collected.amount0);
_decreaseRateLimit(rateLimits, LIMIT_WITHDRAW, token1, pool, collected.amount1);
```

Uni v4 is used for now only on tokens that exchange 1:1. We assume 2 limits are needed because Uni v3 does not have this trait.

- Uni v4 allows calling `decreasePosition` with `liquidityDecrease = 0` to collect fees only, as there is no liquidity validation:

```
function decreasePosition(
    address proxy,
    address rateLimits,
    bytes32 poolId,
    uint256 tokenId,
    uint128 liquidityDecrease, // can be 0
    uint128 amount0Min,
    uint128 amount1Min
)
```

Uni v3 blocks this with an explicit check in `_validateRemoveLiquidityParameters`:

```
require(liquidity != 0 && liquidity <= positionLiquidity, "UniswapV3Lib/liquidity-oob");
```

Maybe on Uni v3 we can consider if `liquidity == 0` we just call `_canCollect`.

Furthermore it could be also considered to implement a separate `collect` or `collectFee` function which only calls `collect` on the `positionManager`.

3.1.2 Current diamond-pau changes that affect Grove

Severity: Medium Risk

Context: `diamond-pau/src/MainnetController.sol`, `diamond-pau/src/ForeignController.sol`, `diamond-pau/src/libraries/ERC4626Lib.sol`, `diamond-pau/src/libraries/AaveLib.sol`, `grove-alm-controller/src/MainnetController.sol`, `grove-alm-controller/src/ForeignController.sol`

Description: The Grove ALM upgrade will not be fully compatible with the current `diamond-pau`, and the team must be aware of the following changes.

`MainnetController` constructor drift:

- Grove v1.8.0:

- `constructor(address admin_, address proxy_, address rateLimits_, address vault_, address psm_, address daiUsds_, address cctp_, address uniswapV3Router_, address uniswapV3PositionManager_).`
- Current diamond-pau:
 - `constructor(address admin_, address proxy_, address rateLimits_, address vault_, address psm_, address daiUsds_, address cctp_).`
- This removes constructor-time wiring for Uniswap V3 router and position manager and replaces it with post-deploy setters:
 - `setUniswapV3SwapRouter(address swapRouter).`
 - `setUniswapV3PositionManager(address manager).`

ForeignController constructor drift:

- Grove v1.8.0:
 - `constructor(address admin_, address proxy_, address rateLimits_, address psm_, address usdc_, address cctp_, address pendleRouter_, address uniswapV3Router_, address uniswapV3PositionManager_).`
- Current diamond-pau:
 - `constructor(address admin_, address proxy_, address rateLimits_, address psm_, address usdc_, address cctp_).`
- This similarly moves part of the integration wiring out of the constructor and into post-deploy setters.

MainnetController removed or incompatible signatures:

- `depositERC4626(address token, uint256 amount)` returns (uint256 shares) no longer exists.
 - Current replacement: `depositERC4626(address token, uint256 amount, uint256 minSharesOut)` returns (uint256 shares).
- `withdrawERC4626(address token, uint256 amount)` returns (uint256 shares) no longer exists.
 - Current replacement: `withdrawERC4626(address token, uint256 amount, uint256 maxSharesIn)` returns (uint256 shares).
- `redeemERC4626(address token, uint256 shares)` returns (uint256 assets) no longer exists.
 - Current replacement: `redeemERC4626(address token, uint256 shares, uint256 minAssetsOut)`
- `prepareUSDeMint(uint256 usdcAmount)` no longer exists.
 - Current replacement: `prepareUSDeMint(uint256 usdcAmount).`
- `prepareUSDeBurn(uint256 usdeAmount)` no longer exists.
 - Current replacement: `prepareUSDeBurn(uint256 usdeAmount).`
- `cooldownAssetsSUSDe(uint256 usdeAmount)` no longer exists.
 - Current replacement: `cooldownAssetsSUSDe(uint256 usdeAmount)` returns (uint256 cooldownShares).
- `cooldownSharesSUSDe(uint256 susdeAmount)` returns (uint256 cooldownAmount) no longer exists.
 - Current replacement: `cooldownSharesSUSDe(uint256 susdeAmount)` returns (uint256 cooldownAssets).
- `unstakeSUSDe()` no longer exists.
 - Current replacement: `unstakeSUSDe().`
- `setUniswapV3TwapSecondsAgo(address pool, uint32 twapSecondsAgo)` no longer exists.
 - Current replacement: `setUniswapV3TWAPSecondsAgo(address pool, uint32 twapSecondsAgo).`

ForeignController removed or incompatible signatures:

- `depositERC4626(address token, uint256 amount)` returns (uint256 shares) no longer exists.
 - Current replacement: `depositERC4626(address token, uint256 amount, uint256 minSharesOut)` returns (uint256 shares).
- `withdrawERC4626(address token, uint256 amount)` returns (uint256 shares) no longer exists.
 - Current replacement: `withdrawERC4626(address token, uint256 amount, uint256 maxSharesIn)` returns (uint256 shares).
- `redeemERC4626(address token, uint256 shares)` returns (uint256 assets) no longer exists.
 - Current replacement: `redeemERC4626(address token, uint256 shares, uint256 minAssetsOut)` returns (uint256 assets).
- `setUniswapV3TwapSecondsAgo(address pool, uint32 twapSecondsAgo)` no longer exists.
 - Current replacement: `setUniswapV3TWAPSecondsAgo(address pool, uint32 twapSecondsAgo)`.
- !!!! Curve-specific entrypoints present in Grove v1.8.0 no longer exist on the current foreign controller:
 - `addLiquidityCurve(address pool, uint256[] depositAmounts, uint256 minLpAmount)` returns (uint256 shares).
 - `removeLiquidityCurve(address pool, uint256 lpBurnAmount, uint256[] minWithdrawAmounts)` returns (uint256[] withdrawnTokens).
 - `swapCurve(address pool, uint256 inputIndex, uint256 outputIndex, uint256 amountIn, uint256 minAmountOut)` returns (uint256 amountOut).

UniswapV3 return-shape incompatibilities on both controllers:

- Grove v1.8.0:
 - `addLiquidityUniswapV3(address pool, uint256 tokenId, UniswapV3Lib.Tick tick, UniswapV3Lib.TokenAmounts target, UniswapV3Lib.TokenAmounts min, uint256 deadline)` returns (uint256 tokenId_, uint128 liquidity_, uint256 amount0_, uint256 amount1_).
 - `removeLiquidityUniswapV3(address pool, uint256 tokenId, uint128 liquidity, UniswapV3Lib.TokenAmounts min, uint256 deadline)` returns (uint256 amount0Collected, uint256 amount1Collected).
- Current diamond-pau:
 - `addLiquidityUniswapV3(address pool, uint256 tokenId, UniswapV3Lib.Ticks ticks, UniswapV3Lib.TokenAmounts target, UniswapV3Lib.TokenAmounts min, uint256 deadline)` returns (uint256 tokenId_, uint128 liquidity_, UniswapV3Lib.TokenAmounts amounts_).
 - `removeLiquidityUniswapV3(address pool, uint256 tokenId, uint128 liquidity, UniswapV3Lib.TokenAmounts min, uint256 deadline)` returns (UniswapV3Lib.TokenAmounts amounts_).

Rate-limit increase / decrease behavior drift on both controllers:

- Grove v1.8.0 treated these unwind paths as one-way consumption of withdraw capacity:
 - `withdrawERC4626(address token, uint256 amount)` returns (uint256 shares) decreased the withdraw-side rate limit only.
 - `redeemERC4626(address token, uint256 shares)` returns (uint256 assets) decreased the withdraw-side rate limit only, using the returned assets.
 - `withdrawAave(address aToken, uint256 amount)` returns (uint256 amountWithdrawn) decreased the withdraw-side rate limit only, using the returned amountWithdrawn.
- Current diamond-pau changes the accounting for the same flows:

- `withdrawERC4626(address token, uint256 amount, uint256 maxSharesIn)` returns (uint256 shares) decreases the withdraw-side rate limit and then increases the deposit-side rate limit by amount.
- `redeemERC4626(address token, uint256 shares, uint256 minAssetsOut)` returns (uint256 assets) decreases the withdraw-side rate limit and then increases the deposit-side rate limit by assets.
- `withdrawAave(address aToken, uint256 amount)` returns (uint256 amountWithdrawn) decreases the withdraw-side rate limit and then increases the deposit-side rate limit by amountWithdrawn.

Recommendation:

- The Curve implementation should be restored, otherwise existing Curve positions may become stranded.
- The rate-limit increase/decrease behavior changes should be explicitly acknowledged and accounted for in the integration.
- The ABI changes should be reflected in any spells, scripts, or off-chain components interacting with the controllers.

Cantina Managed: Partially fixed in recent commits.

3.2 Low Risk

3.2.1 Uniswap V3 and Uniswap V4 asymmetries

Severity: Low Risk

Context: `src/libraries/UniswapV3Lib.sol`, `src/libraries/UniswapV4Lib.sol`

Description: The Uniswap V3 and Uniswap V4 integrations do not currently share a consistent model for withdraw-side limit accounting and fee collection behavior. There are three distinct asymmetries.

1. [PR 38](#) makes V3 exclude fees from withdraw accounting, while V4 still includes them.

The current V4 decrease path explicitly includes fees in the withdraw limit. It snapshots balances before and after `DECREASE_LIQUIDITY`, computes the full normalized balance delta, and documents that fee receipts are included:

- `src/libraries/UniswapV4Lib.sol#L378`.
- `src/libraries/UniswapV4Lib.sol#L391`.
- `src/libraries/UniswapV4Lib.sol#L396`.
- `src/libraries/UniswapV4Lib.sol#L397`.

So after [PR 38](#):

- V3 withdraw limit = principal removed only.
 - V4 withdraw limit = principal removed + fees collected.
2. V4 uses one normalized withdraw limit per pool, while V3 uses two token-side withdraw limits per pool

V4 charges a single pool-level limit:

- `src/libraries/UniswapV4Lib.sol#L397`.

and explicitly relies on a 1:1 value assumption when aggregating `token0` and `token1` into one normalized number:

- `src/libraries/UniswapV4Lib.sol#L389`.

V3 does not make that assumption. Instead, it charges two separate token-side limits:

- `src/libraries/UniswapV3Lib.sol#L438`.
- `src/libraries/UniswapV3Lib.sol#L439`.

3. V4 allows fee-only collection through `decreasePosition(..., liquidityDecrease = 0)`, while V3 blocks it

`UniswapV4Lib.decreasePosition` forwards `liquidityDecrease` without validating it is non-zero:

- `src/libraries/UniswapV4Lib.sol#L156`.
- `src/libraries/UniswapV4Lib.sol#L174`.

So `decreasePosition(..., liquidityDecrease = 0, ...)` can be used as a fee-only collect path if the position manager accepts it.

V3 explicitly forbids this pattern in `_validateRemoveLiquidityParams`:

- `src/libraries/UniswapV3Lib.sol#L791`.
 - `src/libraries/UniswapV3Lib.sol#L819`.
4. V3 and V4 swaps use different rate-limit accounting models. `UniswapV3Lib.swap` performs the swap first, measures the proxy's actual `tokenIn` balance delta, and only then decreases the swap limit by `amountSpent`:
 - `src/libraries/UniswapV3Lib.sol#L300`.
 - `src/libraries/UniswapV3Lib.sol#L306`.

`UniswapV4Lib.swap` decreases the swap limit before execution using `normalizedAmountIn`, and the implementation explicitly notes that this does not account for the net amount of `tokenIn` actually taken:

- `src/libraries/UniswapV4Lib.sol#L218`.
- `src/libraries/UniswapV4Lib.sol#L222`.

So the two integrations already diverge on swap-side accounting as well:

- V3 swap limit = observed post-swap input actually spent.
- V4 swap limit = requested pre-swap input amount, normalized.

Recommendation: Consider documenting this behavior and align both implementations in the next release.

Sky: Fixed in [PR 48](#).

Cantina Managed: Fix verified.

3.2.2 CCTPLib hardcoded `MAX_FEE = 0` has no on-chain update path

Severity: Low Risk

Context: `CCTPLib.sol#L144`

Description: `MAX_FEE` is hardcoded as a constant to `0`, meaning standard (no-fee) CCTP burns. Circle can activate a minimum fee for any route by calling `setMinFee()` on `TokenMessengerV2`. If Circle sets a non-zero minimum fee for standard transfers, every `depositForBurn` call will revert.

```
// TokenMessengerV2.depositForBurn
require(_maxFee >= _calcMinFeeAmount(_amount), "Insufficient max fee");
```

See `TokenMessengerV2.sol#L350` because `MAX_FEE = 0` would be below the minimum.

Currently, all standard transfer routes have `minimumFee = 0` but there is no on-chain path to update `MAX_FEE` without redeploying the controller.

Recommendation: Consider making `MAX_FEE` an admin-configurable storage variable instead of a constant.

Sky: Fixed in [PR 41](#).

Cantina Managed: Fix verified.

3.3 Informational

3.3.1 Rate-limit accounting is inconsistent across request/cancel and unwind flows

Severity: Informational

Context: `diamond-pau/src/libraries/MapleLib.sol`, `diamond-pau/src/libraries/CentrifugeLib.sol`, `diamond-pau/src/libraries/ERC7540Lib.sol`, `diamond-pau/src/libraries/ERC4626Lib.sol`, `diamond-pau/src/libraries/AaveLib.sol`, `diamond-pau/src/libraries/USDSLib.sol`, `diamond-pau/src/libraries/PSMLib.sol`

Description: Rate-limit accounting is inconsistent across integrations. Some flows treat the limit as net exposure and restore capacity when the prior action is unwound or reversed, while others permanently consume limit even when the position change is canceled and no lasting exposure remains.

Examples of symmetric accounting already present in the codebase:

- `ERC4626Lib.withdraw` decreases `LIMIT_WITHDRAW` and then increases `LIMIT_DEPOSIT` by the same asset amount:
 - `src/libraries/ERC4626Lib.sol#L95`.
 - `src/libraries/ERC4626Lib.sol#L109`.
- `ERC4626Lib.redeem` decreases `LIMIT_WITHDRAW` and then increases `LIMIT_DEPOSIT` by the actual assets returned:
 - `src/libraries/ERC4626Lib.sol#L134`.
 - `src/libraries/ERC4626Lib.sol#L135`.
- `AaveLib.withdraw` decreases `LIMIT_WITHDRAW` and then increases `LIMIT_DEPOSIT` by `amountWithdrawn`:
 - `src/libraries/AaveLib.sol#L95`.
 - `src/libraries/AaveLib.sol#L96`.
- `USDSLib.burn` increases the mint-side limit after undoing prior mint-side exposure:
 - `src/libraries/USDSLib.sol#L38`.
 - `src/libraries/USDSLib.sol#L63`.
- `PSMLib.swapUSDCToUSDS` increases the same limit that `swapUSDSToUSDC` decreases:
 - `src/libraries/PSMLib.sol#L56`.
 - `src/libraries/PSMLib.sol#L87`.

By contrast, Maple and Centrifuge request/cancel flows do not restore consumed capacity.

`MapleLib.requestRedemption` decreases `LIMIT_REDEEM` by `convertToAssets(shares)`:

- `src/libraries/MapleLib.sol#L35`.

But `MapleLib.cancelRedemption` only checks that the rate-limit key exists and calls `removeShares`; it does not increase the limit back:

- `src/libraries/MapleLib.sol#L54`.
- `src/libraries/MapleLib.sol#L61`.

The same asymmetry exists in the Centrifuge flow, which is split across `ERC7540Lib` and `CentrifugeLib`:

- `ERC7540Lib.requestDeposit` decreases `LIMIT_DEPOSIT`:
 - `src/libraries/ERC7540Lib.sol#L56`.
- `ERC7540Lib.requestRedeem` decreases `LIMIT_REDEEM`:
 - `src/libraries/ERC7540Lib.sol#L80`.
- `CentrifugeLib.cancelDepositRequest`, `claimCancelDepositRequest`, `cancelRedeemRequest`, and `claimCancelRedeemRequest` only check that the corresponding key exists and do not restore capacity:
 - `src/libraries/CentrifugeLib.sol#L84`.

- src/libraries/CentrifugeLib.sol#L94.
- src/libraries/CentrifugeLib.sol#L106.
- src/libraries/CentrifugeLib.sol#L116.

This makes rate-limit semantics integration-specific instead of policy-driven.

Recommendation: Standardize rate-limit accounting across all integrations.

Sky: Acknowledged.

Cantina Managed: Acknowledged.

3.3.2 Current diamond-pau changes that affect Spark

Severity: Informational

Context: Spark-MainnetController, Spark-ForeignController, PAU-MainnetController, PAU-ForeignController

Description: We compared the current dev branch in Spark and diamond-pau for differences.

- Spark: `sparkdotfi/spark-alm-controller` dev branch (0fee4b5b1bc4de7da1e7aa5b91a98375ef6849a2)
- PAU: `sky-ecosystem/diamond-pau` dev branch (282ac6c1d7d6fa61bd01e8c7e1f04642907995c7)

MainnetController:

Spark: **117** functions | PAU: **156** functions | Shared: **117** | Removed: **0** | Added: **39**

- Removed selectors: None.
- New selectors (in PAU, not in Spark):

```
+ 092244cf  setPendleRouter(address)
+ 18ae59d7  centrifugeRecipients(uint16)
+ 19768e89  LIMIT_UNISWAP_V3_DEPOSIT()
+ 206aeab3  pendleRouter()
+ 256c49f1  setUniswapV3AddLiquidityUpperTickBound(address,int24)
+ 2941903c  LIMIT_PENDLE_PT_REDEEM()
+ 2c76d7a6  uniswapV3Router()
+ 2fa4abea  merklDistributor()
+ 342a30c3  uniswapV3PositionManager()
+ 3e34ecfc  setUniswapV3AddLiquidityLowerTickBound(address,int24)
+ 4390e9dd  claimRedeemERC7540(address)
+ 43e7ba06  uniswapV3PoolParams(address)
+ 4d8f42b2  addLiquidityUniswapV3(address,uint256,(int24,int24),(uint256,uint256),(uint2
↳ 56,uint256),uint256)
+ 558e0a77  claimCentrifugeCancelDepositRequest(address)
+ 5ad034a6  toggleOperatorMerkle(address)
+ 5bbl9a74  cancelCentrifugeDepositRequest(address)
+ 5e885b45  setUniswapV3SwapRouter(address)
+ 6111988c  transferUSDCtoCTP(uint256,uint256,uint32)
+ 62facbf1  removeLiquidityUniswapV3(address,uint256,uint128,(uint256,uint256),uint256)
+ 7bac30dd  redeemPendlePT(address,uint256,uint256)
+ 82fc048e  setUniswapV3PoolMaxTickDelta(address,uint24)
+ 8603290c  setUniswapV3TWAPSecondsAgo(address,uint32)
+ 894f1160  LIMIT_UNISWAP_V3_SWAP()
+ 9ec0d57e  setCentrifugeRecipient(uint16,bytes32)
+ a0b0c6af  claimCentrifugeCancelRedeemRequest(address)
+ a5d4415c  LIMIT_UNISWAP_V3_WITHDRAW()
+ b0230f8a  LIMIT_CENTRIFUGE_TRANSFER()
+ b2b094a0  transferSharesCentrifuge(address,uint128,uint16)
+ b7a589d2  setUniswapV3PositionManager(address)
+ bc206c37  swapUniswapV3(address,address,uint256,uint256,uint24)
+ c07793ad  LIMIT_7540_DEPOSIT()
+ c2be3706  requestDepositERC7540(address,uint256)
+ c67c9ea9  setCTPMaxFeeCap(uint256)
+ c95c29d9  requestRedeemERC7540(address,uint256)
+ d0a77054  LIMIT_7540_REDEEM()
```

```

+ dbfba959 setMerkleDistributor(address)
+ e08471fc claimDepositERC7540(address)
+ e604ddab cancelCentrifugeRedeemRequest(address)
+ fda69124 cctpMaxFeeCap()

```

- Changed signatures (same name, different selector or overload added):

transferUSDCToCCTP

```

- bcd7e46c transferUSDCToCCTP(uint256,uint32)
+ 6111988c transferUSDCToCCTP(uint256,uint256,uint32)
+ bcd7e46c transferUSDCToCCTP(uint256,uint32)

```

ForeignController:

Spark: **48** functions | PAU: **93** functions | Shared: **48** | Removed: **0** | Added: **45**

- Removed selectors: None.
- New selectors (in PAU, not in Spark):

```

+ 072ef76d removeLiquidityCurve(address,uint256,uint256[])
+ 092244cf setPendleRouter(address)
+ 0a04920d LIMIT_CURVE_WITHDRAW()
+ 18ae59d7 centrifugeRecipients(uint16)
+ 19768e89 LIMIT_UNISWAP_V3_DEPOSIT()
+ 1eae443 swapCurve(address,uint256,uint256,uint256,uint256)
+ 206aeab3 pendleRouter()
+ 256c49f1 setUniswapV3AddLiquidityUpperTickBound(address,int24)
+ 2941903c LIMIT_PENDLE_PT_REDEEM()
+ 2c76d7a6 uniswapV3Router()
+ 2fa4abea merkleDistributor()
+ 342a30c3 uniswapV3PositionManager()
+ 38fe30b0 LIMIT_CURVE_DEPOSIT()
+ 3e34ecfc setUniswapV3AddLiquidityLowerTickBound(address,int24)
+ 4390e9dd claimRedeemERC7540(address)
+ 43e7ba06 uniswapV3PoolParams(address)
+ 4d8f42b2 addLiquidityUniswapV3(address,uint256,(int24,int24),(uint256,uint256),(uint2
↳ 56,uint256),uint256)
+ 533a5901 addLiquidityCurve(address,uint256[],uint256)
+ 5364049c LIMIT_CURVE_SWAP()
+ 558e0a77 claimCentrifugeCancelDepositRequest(address)
+ 5ad034a6 toggleOperatorMerkle(address)
+ 5bbl974 cancelCentrifugeDepositRequest(address)
+ 5e885b45 setUniswapV3SwapRouter(address)
+ 6111988c transferUSDCToCCTP(uint256,uint256,uint32)
+ 62facbf1 removeLiquidityUniswapV3(address,uint256,uint128,(uint256,uint256),uint256)
+ 7bac30dd redeemPendlePT(address,uint256,uint256)
+ 82fc048e setUniswapV3PoolMaxTickDelta(address,uint24)
+ 8603290c setUniswapV3TWAPSecondsAgo(address,uint32)
+ 894f1160 LIMIT_UNISWAP_V3_SWAP()
+ 9ec0d57e setCentrifugeRecipient(uint16,bytes32)
+ a0b0c6af claimCentrifugeCancelRedeemRequest(address)
+ a5d4415c LIMIT_UNISWAP_V3_WITHDRAW()
+ b0230f8a LIMIT_CENTRIFUGE_TRANSFER()
+ b2b094a0 transferSharesCentrifuge(address,uint128,uint16)
+ b7a589d2 setUniswapV3PositionManager(address)
+ bc206c37 swapUniswapV3(address,address,uint256,uint256,uint24)
+ c07793ad LIMIT_7540_DEPOSIT()
+ c2be3706 requestDepositERC7540(address,uint256)
+ c67c9ea9 setCCTPMaxFeeCap(uint256)
+ c95c29d9 requestRedeemERC7540(address,uint256)
+ d0a77054 LIMIT_7540_REDEEM()
+ dbfba959 setMerkleDistributor(address)
+ e08471fc claimDepositERC7540(address)
+ e604ddab cancelCentrifugeRedeemRequest(address)
+ fda69124 cctpMaxFeeCap()

```

- Changed signatures (same name, different selector or overload added):

transferUSDCToCCTP

```
- bcd7e46c transferUSDCToCCTP(uint256,uint32)
+ 6111988c transferUSDCToCCTP(uint256,uint256,uint32)
+ bcd7e46c transferUSDCToCCTP(uint256,uint32)
```

Return Type Differences:

- MainnetController: No return type differences.
- ForeignController: No return type differences.

Summary:

- MainnetController: Spark 117 | PAU 156 | Shared 117 | Removed 0 | Added 39
- ForeignController: Spark 48 | PAU 93 | Shared 48 | Removed 0 | Added 45

Recommendation: Document the changes between Spark and diamond-pau.

Sky: Acknowledged.

Cantina Managed: Acknowledged.

3.3.3 UniswapV3 removeLiquidity requires an active withdraw limit even for the zero-output side

Severity: Informational

Context: [src/libraries/UniswapV3Lib.sol](#), [src/RateLimits.sol](#)

Description: `UniswapV3Lib.removeLiquidity` always decreases the withdraw-side rate limit for both pool tokens after `collect`, regardless of whether one side is actually zero:

- [src/libraries/UniswapV3Lib.sol#L438](#).
- [src/libraries/UniswapV3Lib.sol#L439](#).

For an out-of-range Uniswap V3 position or for a single-side position, all removable liquidity sits on only one side. In that case, `collect` can return:

- `collected.amount0 > 0` and `collected.amount1 == 0`, or...
- `collected.amount0 == 0` and `collected.amount1 > 0`

Now, the library calls `_decreaseRateLimit(...)` for the zero-output side which can revert if the limit is not set for that side, the `RateLimits.triggerRateLimitDecrease` does not treat a zero-amount decrease as a no-op when the key is unset or disabled. It first requires:

- [src/RateLimits.sol#L97](#)

```
require(maxAmount > 0, "RateLimits/zero-maxAmount");
```

So `triggerRateLimitDecrease(key, 0)` is only valid if that rate-limit key already exists and has `maxAmount > 0`.

Recommendation: Treat zero-amount withdraw-side decreases in `UniswapV3Lib.removeLiquidity` as no-ops instead of always calling `triggerRateLimitDecrease`.

Sky: Acknowledged. For a two asset pool we expect rate limits to always be non zero for both assets.

Cantina Managed: Acknowledged.

3.3.4 CurveLib.swap minAmountOut check is less conservative due to additional truncation

Severity: Informational

Context: [diamond-pau/src/libraries/CurveLib.sol](#), [grove-alm-controller/src/libraries/CurveLib.sol](#)

Description: The current `CurveLib.swap minAmountOut` check is less conservative than the previous implementation because it performs additional intermediate truncation before applying `maxSlippages[pool]`.

Current `diamond-pau` logic:

- `src/libraries/CurveLib.sol#L89`.
- `src/libraries/CurveLib.sol#L93`.

```
uint256 valueIn          = _toNormalizedAmount(amountIn, rates[inputIndex]);
uint256 equivalentAmountOut = _fromNormalizedAmount(valueIn, rates[outputIndex]);
require(
    minAmountOut >= equivalentAmountOut * maxSlippages[pool] / 1e18,
    "CurveLib/min-amount-not-met"
);
```

Fully expanded, this is:

```
floor( floor( floor(amountIn * rates[inputIndex] / 1e18) * 1e18 / rates[outputIndex] ) *
→ maxSlippages[pool] / 1e18 )
```

The previous Grove implementation applied `maxSlippage` in the combined expression and explicitly documented that the multiplication was ordered first to avoid precision loss:

- `src/libraries/CurveLib.sol#L98`.
- `src/libraries/CurveLib.sol#L106`.

```
uint256 minimumMinAmountOut = params.amountIn
    * rates[params.inputIndex]
    * params.maxSlippage
    / rates[params.outputIndex]
    / 1e18;
```

Fully expanded, this is:

```
floor( floor(amountIn * rates[inputIndex] * maxSlippage / rates[outputIndex]) / 1e18 )
```

Because the current implementation truncates once when converting `amountIn` into normalized value, again when converting that value back into output-token amount, and only then applies `maxSlippages[pool]`, it can produce a slightly lower required `minAmountOut` than the previous formula.

Recommendation: Restore the previous precision-preserving ordering for the `minAmountOut` bound so `maxSlippages[pool]` is applied before the extra truncation steps.

Sky: We opted to make the code easier to understand here, acknowledge small rounding error got introduced.

Cantina Managed: Acknowledged.

3.3.5 LayerZeroLib: no refund for overpaid nativeFee in ETH

Severity: Informational

Context: `LayerZeroLib.sol`

Description: In `transfer()`, `fee.nativeFee` is forwarded to the LayerZero endpoint via `doCallWithValue`, but if `msg.value > fee.nativeFee`, the difference remains in the controller. Pre-existing from v10. Enforcing `msg.value == fee.nativeFee` is not advisable because the relayer estimates fees off-chain and LayerZero DVN/executor pricing can change between blocks.

Recommendation: Consider refunding `msg.value - fee.nativeFee` back to the caller after the `send()` call.

Sky: Fixed in PR 43.

Cantina Managed: Fix verified.

3.3.6 Minor issues, Typos & Documentation

Severity: Informational

Context: (See each case below)

Description:

1. `WSTETHLib.sol#L7`: The local ERC20 interface is named `IERC20ike`, which appears to be a typo of `IERC20Like`.
2. `USDSLib.sol#L44`, `src/libraries/USDSLib.sol#L66`: The comment says `Not need for ApprobeLib`, which contains both a grammar issue and a typo in `ApproveLib`.
3. `MainnetController.sol#L1081`: The comment uses `cancelation`, while the standard spelling is `cancellation`.
4. `UniswapV3Lib.sol#L238-L250`: `UniswapV3Lib.setTWAPSecondsAgo` only enforces an upper bound (~68 years) but accepts values as low as 1, which would reduce the TWAP to effectively a spot price and weaken manipulation resistance in `swap()` and `addLiquidity()`. Consider enforcing a minimum.
5. `UniswapV3Lib.sol#L177`: The comment references the `Uniswap v4 TickMath` for `MIN_TICK` and `MAX_TICK`. While the values are identical, this is a Uniswap v3 library and should reference the `Uniswap v3 TickMath` instead.
6. `UniswapV4Lib.sol#L236-L239`: The `swap()` slippage check assumes the two tokens in the pool are valued equally (e.g., stablecoin pairs). This assumption is not good documented. Consider adding `natspec` to the function header clarifying this constraint, so that other stars reusing these libs are aware of the limitation.
7. `CCTPLib.sol#L106`: The burn loop in `transfer()` has no sanity check for `burnLimit > 0`. If `burnLimitPerMessage()` returns 0 (misconfigured CCTP TokenMinter), the while loop becomes infinite and consumes all gas. Consider adding `require(burnLimit > 0)`.
8. `ApproveLib.sol#L12`: `ApproveLib` only has internal functions and is used by other libraries, it won't become its own facet. Consider moving it to a subfolder like `libraries/internal/` to clearly separate them from libraries that will become facets in the diamond upgrade.
9. `LayerZeroLib.sol#L105`: If `msg.value > fee.nativeFee`, the excess ETH remains locked in the controller. Enforcing `msg.value == fee.nativeFee` is not advisable because the relayer can change between blocks. Consider refunding `msg.value - fee.nativeFee` back to the caller after the `send()` call.
10. `WEETHModule.sol#L53`: Some facets depend on per-star individual contracts (e.g., `WEETHModule`, `OTCBuffer`) that must be deployed separately. Consider documenting which facets require these extra deployments when onboarding new stars.
11. `WEETHModule.sol#L88`: If the `DEFAULT_ADMIN_ROLE` of `WEETHModule` should always match the `ALMProxy`'s admin, consider verifying in `initialize()` that the passed admin matches `almProxy.DEFAULT_ADMIN_ROLE` to prevent misconfiguration during deployment.
12. `MainnetController.sol#L7`: The `spark-address-registry` dependency is Spark-specific. For shared diamond facets, consider migrating to a general-purpose address registry that only includes common addresses (e.g., WETH) rather than star-specific ones.
13. `MainnetController.sol#L117`: `buffer` is dead storage. set in the constructor but never read. `USDSLib` calls `vault.buffer()` directly each time. Consider removing `buffer`, the `IVaultLike` interface, and the constructor assignment.
14. `MainnetController.sol#L196`: `setMintRecipient` is too generic. it doesn't indicate which protocol it belongs to. Consider renaming to `setCCTPMintRecipient` for consistency with the naming convention used by other setters.
15. `MainnetController.sol#L13`: `ApproveLib` is not used in the `MainnetController` Use this:
16. `WEETHModule.sol`: Standardize the naming of all buffer-like modules under a single explicit convention. Any component that serves a buffer role should be named and documented as a buffer component so its purpose and required usage are unambiguous.

17. `UniswapV3OracleLib.sol`: The errors inside this library should be prefixed with `UniswapV3OracleLib/*`.
18. `UniswapV3OracleLib.sol`: The two `FullMath` and `TickMath` imports can be removed.
19. `UniswapV3OracleLib.sol`: In `CCTPLib` we use this approach to assign `mintRecipients`:
`emit MintRecipientSet(destinationDomain, mintRecipients[destinationDomain] = recipient)`
but in `LayerZeroLib` you don't. Consider standardizing this.
20. `CurveLib.sol`: The `coins` function should be declared a `view`.

Recommendation: Consider fixing the recommendations mentioned above.

Sky: Addressed 1, 2 and 3 in [PR 40](#). Others are acknowledged.

Cantina Managed: Partially fixed.